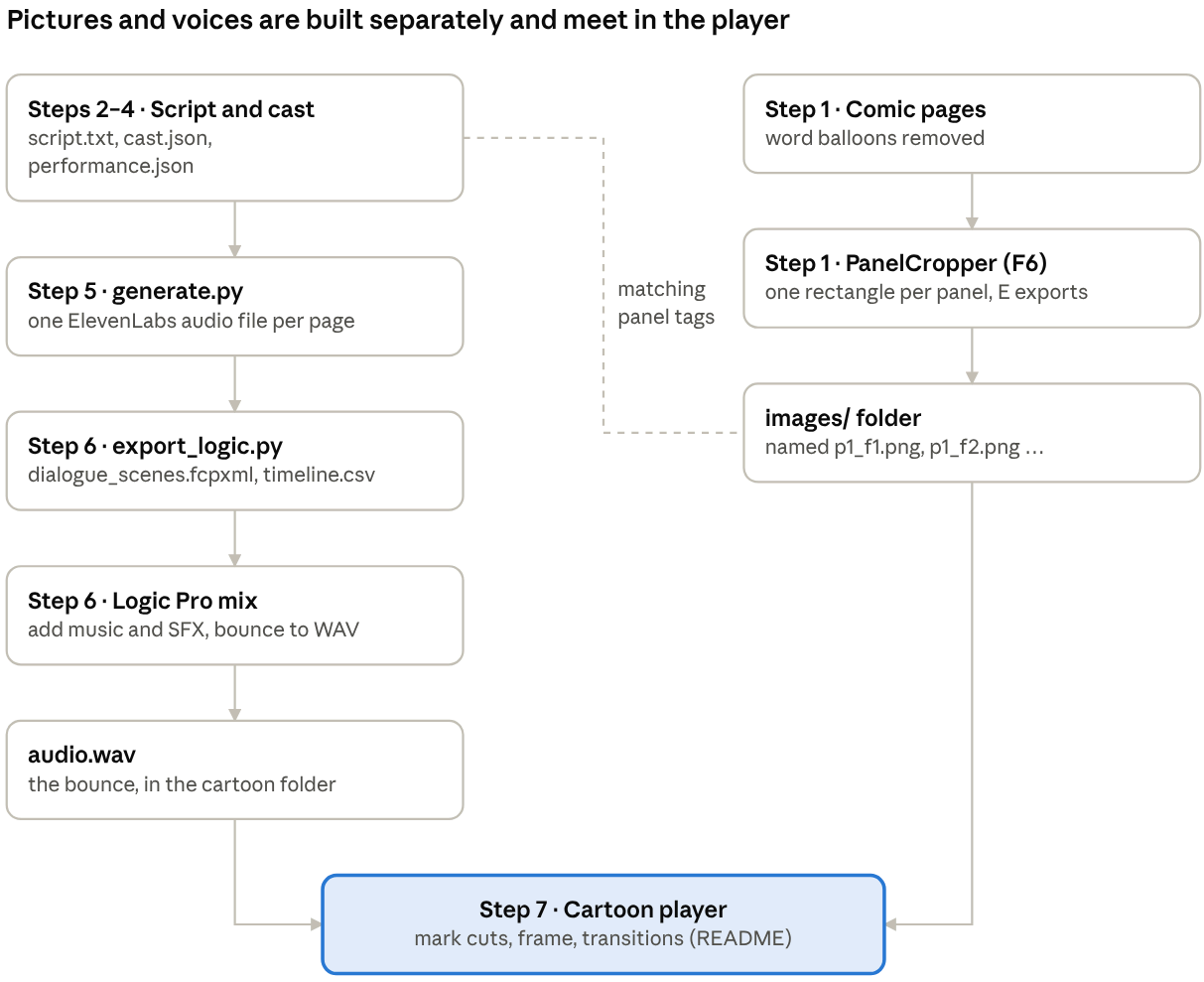
CartoonMaker Quick Start

Oct 23, 2026 · @Michael

Overview

A cartoon is one folder, `cartoons/<name>/`, holding its panel images, one soundtrack (`audio.wav`) and `cartoon.json`. The pictures take an hour; the voice track is the real work, and it runs through two Python scripts that sit in the cartoon folder: `generate.py` (ElevenLabs voices) and `export_logic.py` (layout for Logic Pro).

This guide stops where the player takes over. Marking cuts, camera moves and transitions are covered in the README.



CartoonMaker pipeline · 7 steps, two tracks

The panel tag (`p2_f5`) is the thread between the two tracks: it names the image file and labels each script line.

What you need installed

- Godot 4.7 (the project uses the Compatibility renderer)
- Python 3.9 or later, with `requests` : `pip3 install requests`
- ffmpeg and ffprobe on your PATH: `brew install ffmpeg`
- An ElevenLabs account and API key (Profile > API Keys on elevenlabs.io)
- Logic Pro, for the mix

Two things to know up front

- The scripts live in `cartoons/captain_marvel_subconscious/` , which is the reference for the current workflow. Copy `generate.py` and `export_logic.py` into each new cartoon folder; they work on whatever folder they sit in.
- Only `audio.wav` is in `.gitignore` . The generated page files (`clips/`) and the Logic export (`logic/`) are committed, so git keeps every take; keep your own copy of the bounce.

Step 1: Gather your images

Every panel ends up in `cartoons/<name>/images/` named `p<page>_f<frame>.png` . That name is the key the whole audio pipeline hangs on: the script tags each line with the same `p<page>_f<frame>` , so each voice line knows which picture it belongs to.

1. **Create the cartoon.** Run the project, type a folder name in the picker and press **Create**. This makes `cartoons/<name>/images/` .
2. **Prepare the pages.** Remove the word balloons first. The cropper cuts what it is given and does not clean anything.
3. **Cut the panels (if needed).** Open `tools/panel_cropper.tscn` and press **F6**.
   a. Pick your cartoon at the top left.
   b. **Pages folder...** points at the cleaned pages (anywhere on disk).
   c. Draw a rectangle around each panel in reading order. Fix the order with `,` and `.` .
   d. Check the **Page** box. It reads the last number in the file name.
   e. Press `E` to export the page. Panels are written as full-resolution PNGs.
4. **Already have panels?** Rename them to `p<page>_f<frame>` and drop them in `images/` . Zero padding is optional.

Rectangles are saved in `crops.json`, so a touched-up page can be re-exported with identical crops. Delete any old panel files the status line warns about.

Step 2: Write the tagged script

Create `script.txt` in the cartoon folder with one line per balloon or caption. Only lines that start with a panel tag are read; titles, cast lists and page headers are ignored, so annotate freely.

```
[p2_f5] NARRATOR: —it's Captain Marvel to the rescue!
[p2_f5] PROFESSOR SMITH: HELP! Help! Help!
[p2_f5] CAPTAIN MARVEL: Coming!
[p8_f1] BILLY BATSON (thinking): GOSH! What a costume!
```

- **Format:** `[p<page>_f<frame>] SPEAKER (optional qualifier): text`.
- **Line IDs** come from the order within each panel: the three `p2_f5` lines above become `p2_f5_01`, `p2_f5_02`, `p2_f5_03`. Every output file and every retake uses these IDs.
- **CAPS** for words lettered bold in the original. The voices lean on them for emphasis.
- **Speaker names** must match a key in `cast.json`. Case and punctuation are ignored, so `Mr. Morris` matches `MR. MORRIS`.
- Inserting a line in the middle of a panel renumbers the lines after it. That is safe: tags in `performance.json` are re-matched by their words.

Step 3: Cast the voices

`cast.json` maps each speaker to an ElevenLabs voice. Add any voices you want to your ElevenLabs library first, then list their IDs:

```
export ELEVENLABS_API_KEY=your_key_here
python3 generate.py --list-voices
```

Then fill in one entry per speaker:

```
{
  "_defaults": {
    "mode": "dialogue",
    "output_format": "mp3_44100_128",
    "dialogue": { "model_id": "eleven_v4", "stability": 0.5 }
  },
  "NARRATOR":       { "voice_id": "EapdDtSsMC291mjfSNe7", "voice_name":
"Peter" },
  "CAPTAIN MARVEL": { "voice_id": "d8PSLVicWEH7zbII5JEI", "voice_name":
"Kory" }
}
```

- **`voice_name`** goes into every clip's file name, so you can tell takes apart in Finder and Logic.
- **`dialogue`** settings are optional; those shown are the built-in defaults. Add `"seed": 1234` for more repeatable takes.
- **`voice_settings`** and **`qualifiers`** (per-character stability, style and so on) only apply in `line` mode. The default dialogue mode ignores them; direct delivery there with tags (Step 4).
- Every speaker in the script needs a voice before a real run.

Audition the cast before spending credits on the whole script:

```
python3 generate.py script.txt --voice-test
```

This writes one sample per character, using one of their longer lines, to `clips/_voicetest/`. Swap voices in `cast.json` and rerun until the cast sounds right.

Step 4: Direct the performance (optional)

`performance.json` is a copy of the script with ElevenLabs audio tags added. Tags in square brackets steer delivery and are not spoken. Without this file, lines are read untagged.

```
{
  "lines": {
    "p1_f3_02": {
      "character": "PROFESSOR SMITH",
      "text": "HELP! HELP! SAVE ME!",
      "performed": "[distant, terrified shouting] HELP! HELP! SAVE ME!"
    }
  }
}
```

- **`text`** must match the script line word for word. Edit the script later and that line falls back to untagged until you update `text` here too.
- **`performed`** must contain the same words plus tags. If you change the words themselves, the script prints a warning.
- Tags work in dialogue mode only; `line` mode always sends the plain script text.
- A quick way to draft it: give Claude `script.txt` and a short character sketch, and ask for tags in this format. Then tune by ear.
- A change to a line's tags counts as a change to the line, so that page is regenerated on the next run.

Step 5: Generate the voice tracks

Each page is performed as one scene through ElevenLabs Text to Dialogue, so the characters answer each other with natural timing. The current workflow keeps each page as one audio file (`--scenes-only`) rather than cutting it into per-line clips. Step 6 lays those page files out for Logic.

```
cd cartoons/<name>
export ELEVENLABS_API_KEY=your_key_here
python3 generate.py script.txt --scenes-only --dry-run   # plan only, no API
calls, no credits
python3 generate.py script.txt --scenes-only --page 1    # try one page first
python3 generate.py script.txt --scenes-only             # every page with a
new or changed line
```

Pass `--scenes-only` on every run; without it the script also cuts per-line clips. Pages whose lines did not change are skipped, so rerunning is cheap. A page over about 1,800 characters or 10 voices is split at a panel boundary into two scenes (`p03_c1` , `p03_c2`).

What it writes

File	What it is
`clips/_scenes/p01_t1.mp3`	Page 1 as performed, take 1
`clips/_alignment/p01_t1.json`	ElevenLabs timing for that take
`manifest.json`	Every line and take, and where each line sits in its page file; the record `export_logic.py` reads
`manifest.csv`	One row per line, for a quick look in Numbers
`scenes.csv`	Start and end of each line inside its page file

Check the `status` column of `manifest.csv`: `ok` is done, `pending` has never been generated, `stale` has an old take but the line has since changed, `no_voice` needs a `cast.json` entry.

Retakes

To	Run
Redo a page	`python3 generate.py script.txt --scenes-only --retake p3_f5_02` (any line ID on that page)
Redo several pages	`--retake p3_f5_02,p7_f1_01`
Go back to an older take	add `"pin_take": N` to each line of that page in `manifest.json`, where N is the line's take number in its `takes` list

A retake re-performs the whole page and costs that page's characters. Old page files are never deleted. Skip `--single` in this workflow: it makes a per-line clip, which `export_logic.py --scenes` cannot place.

Step 6: Lay out, mix and bounce in Logic

`export_logic.py --scenes` puts the page files end to end on one track and hands them to Logic as Final Cut Pro XML:

```
python3 export_logic.py --scenes
python3 export_logic.py --scenes --lead-in 2 --page-gap 1.5   # with your own
spacing
```

Option	Default	Sets
`--lead-in`	1.0 s	Silence before page 1
`--page-gap`	0.5 s	Silence between pages

The pauses inside a page are ElevenLabs' own timing, so `--line-gap` and `--panel-gap` do not apply here. When every line is a page take, the script picks this mode even without `--scenes`.

What it writes

- `logic/dialogue_scenes.fcpxml`, the file you import
- `logic/scenes/p01_t1.wav` and so on: 48 kHz / 24-bit Broadcast WAVs stamped with their timeline position
- `timeline.json` / `timeline.csv`: the start and end of every line and every panel on the full timeline

Layout is recomputed on every run. A retake that comes back longer or shorter moves every page after it. Re-export after retakes, then re-import into Logic.

In Logic

1. **File > Import > Final Cut Pro XML** and pick `logic/dialogue_scenes.fcpxml`. The pages land on one Dialogue track, with a marker at every panel.
2. Add music and sound effects (`audio/effects/` holds the library) and mix.
3. Bounce to an uncompressed WAV and save it as `cartoons/<name>/audio.wav`.

If you move pages in Logic, `timeline.csv` no longer matches the bounce. That only matters if you use it for marking (Step 7).

Step 7: Bring it into the player

With `audio.wav` in the cartoon folder, run the project and pick the cartoon. Put the WAV in place before the first open, so the new `cartoon.json` points at it. If `cartoon.json` already exists, set its `audio_path` to `"audio.wav"` with the player closed.

- In Godot's Import tab, leave the WAV's compression **Disabled**, or the waveform will not show.
- **Scaffold** or **Start empty**, then mark, frame and transition as the README describes.
- `timeline.csv` lists when each panel's first line starts. Use it as a cue sheet while you tap `M`, or paste the panel times into the `start` fields of `cartoon.json`. Both work only if you did not move dialogue in Logic.
- Re-bounced the mix? Overwrite `audio.wav`. Marker times are plain seconds, so they stay valid as long as the dialogue did not move.

Troubleshooting

Message or symptom	Fix
`Set ELEVENLABS_API_KEY first`	`export` the key in the same Terminal window, or add it to `~/.zshrc`
`Assign voices in cast.json before generating: …`	Add the listed speakers to `cast.json`, spelled as in the script
`performance.json: no tags found for …`	The script line changed; update its `text` in `performance.json`
`performance.json: … changes the words, not just tags`	Make `performed` match the script's words exactly, apart from tags
`… missing from the scene audio — retake with --single`	ElevenLabs dropped a line from the page; retake the page with `--scenes-only --retake` (not `--single`)
`… has no scene take — run generate.py --scenes-only first`	That line was last generated without `--scenes-only`; retake its page with `--scenes-only`
`EL API kept rate-limiting / failing`	Wait a few minutes and rerun; finished pages are not redone
No waveform in the player	Bounce as WAV, and set compression to Disabled in the Import tab
`ffmpeg` or `ffprobe` not found	`brew install ffmpeg`